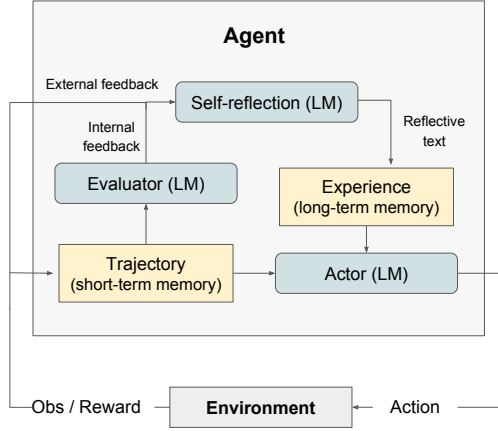




Algorithm 1 Reinforcement via self-reflection

```

Initialize Actor, Evaluator, Self-Reflection:
 $M_a, M_e, M_{sr}$ 
Initialize policy  $\pi_\theta(a_i|s_i), \theta = \{M_a, mem\}$ 
Generate initial trajectory using  $\pi_\theta$ 
Evaluate  $\tau_0$  using  $M_e$ 
Generate initial self-reflection  $sr_0$  using  $M_{sr}$ 
Set  $mem \leftarrow [sr_0]$ 
Set  $t = 0$ 
while  $M_e$  not pass or  $t < \text{max trials}$  do
    Generate  $\tau_t = [a_0, o_0, \dots, a_i, o_i]$  using  $\pi_\theta$ 
    Evaluate  $\tau_t$  using  $M_e$ 
    Generate self-reflection  $sr_t$  using  $M_{sr}$ 
    Append  $sr_t$  to  $mem$ 
    Increment  $t$ 
end while
return

```

Figure 2: (a) Diagram of Reflexion. (b) Reflexion reinforcement algorithm

Actor The Actor is built upon a large language model (LLM) that is specifically prompted to generate the necessary text and actions conditioned on the state observations. Analogous to traditional policy-based RL setups, we sample an action or generation, a_t , from the current policy π_θ at time t , receive an observation from the environment o_t . We explore various Actor models, including Chain of Thought [26] and ReAct [30]. These diverse generation models allow us to explore different aspects of text and action generation within the Reflexion framework, providing valuable insights into their performance and effectiveness. In addition, we also add a memory component mem that provides additional context to this agent. This adaption was inspired by Brooks et al. [3], who suggest a policy iteration approach using in-context learning. Details on how this is populated are provided below.

Evaluator The Evaluator component of the Reflexion framework plays a crucial role in assessing the quality of the generated outputs produced by the Actor. It takes as input a generated trajectory and computes a reward score that reflects its performance within the given task context. Defining effective value and reward functions that apply to semantic spaces is difficult, so we investigate several variants of the Evaluator model. For reasoning tasks, we explore reward functions based on exact match (EM) grading, ensuring that the generated output aligns closely with the expected solution. In decision-making tasks, we employ pre-defined heuristic functions that are tailored to specific evaluation criteria. Additionally, we experiment with using a different instantiation of an LLM itself as an Evaluator, generating rewards for decision-making and programming tasks. This multi-faceted approach to Evaluator design allows us to examine different strategies for scoring generated outputs, offering insights into their effectiveness and suitability across a range of tasks.

Self-reflection The Self-Reflection model instantiated as an LLM, plays a crucial role in the Reflexion framework by generating verbal self-reflections to provide valuable feedback for future trials. Given a sparse reward signal, such as a binary success status (success/fail), the current trajectory, and its persistent memory mem , the self-reflection model generates nuanced and specific feedback. This feedback, which is more informative than scalar rewards, is then stored in the agent’s memory (mem). For instance, in a multi-step decision-making task, when the agent receives a failure signal, it can infer that a specific action a_i led to subsequent incorrect actions a_{i+1} and a_{i+2} . The agent can then verbally state that it should have taken a different action, a'_i , which would have resulted in a'_{i+1} and a'_{i+2} , and store this experience in its memory. In subsequent trials, the agent can leverage its past experiences to adapt its decision-making approach at time t by choosing action a'_i . This iterative process of trial, error, self-reflection, and persisting memory enables the agent to rapidly improve its decision-making ability in various environments by utilizing informative feedback signals.

Memory Core components of the Reflexion process are the notion of short-term and long-term memory. At inference time, the Actor conditions its decisions on short and long-term memory, similar

to the way that humans remember fine-grain recent details while also recalling distilled important experiences from long-term memory. In the RL setup, the trajectory history serves as the short-term memory while outputs from the Self-Reflection model are stored in long-term memory. These two memory components work together to provide context that is specific but also influenced by lessons learned over several trials, which is a key advantage of Reflexion agents over other LLM action choice works.

The Reflexion process Reflexion is formalized as an iterative optimization process in 1. In the first trial, the Actor produces a trajectory τ_0 by interacting with the environment. The Evaluator then produces a score r_0 which is computed as $r_t = M_e(\tau_0)$. r_t is only a scalar reward for trial t that improves as task-specific performance increases. After the first trial, to amplify r_0 to a feedback form that can be used for improvement by an LLM, the Self-Reflection model analyzes the set of $\{\tau_0, r_0\}$ to produce a summary sr_0 which is stored in the memory *mem*. sr_t is a verbal experience feedback for trial t . The Actor, Evaluator, and Self-Reflection models work together through trials in a loop until the Evaluator deems τ_t to be correct. As mentioned in 3, the memory component of Reflexion is crucial to its effectiveness. After each trial t , sr_t is appended *mem*. In practice, we bound *mem* by a maximum number of stored experiences, Ω (usually set to 1-3) to adhere to max context LLM limitations.

4 Experiments

We evaluate various natural language RL setups on decision-making, reasoning, and code generation tasks. Specifically, we challenge an agent to perform search-based question answering on HotPotQA [28], multi-step tasks in common household environments in AlfWorld [24], and code writing tasks in competition-like environments with interpreters and compilers in HumanEval [6], MBPP [2], and LeetCodeHard, a new benchmark. Most notably, Reflexion improves performance over strong baselines by 22% in AlfWorld, 20% in HotPotQA, and 11% on HumanEval.

4.1 Sequential decision making: ALFWorld

AlfWorld is a suite of text-based environments that challenge an agent to solve multi-step tasks in a variety of interactive environments based on TextWorld [8]. Following Yao et al. [30], we run the agent in 134 AlfWorld environments across six different tasks, including finding hidden objects (e.g., finding a spatula in a drawer), moving objects (e.g., moving a knife to the cutting board), and manipulating objects with other objects (e.g., chilling a tomato in the fridge). We use ReAct [30] as the action generator as Yao et al. [30] has shown success in long trajectory decision-making using explicit intermediate thoughts. AlfWorld tasks naturally require a self-evaluation step as the environment can only signal if a task is complete. To achieve fully autonomous behavior, we implement two self-evaluation techniques: natural language classification using an LLM and a hand-written heuristic. The heuristic is simple: if the agent executes the same action and receives the same response for more than 3 cycles, or if the number of actions taken in the current environment exceeds 30 (inefficient planning), we self-reflect. In the baseline runs, if self-reflection is suggested, we skip the self-reflection process, reset the environment, and start a new trial. In the Reflexion runs, the agent uses self-reflection to find its mistake, update its memory, reset the environment, and start a new trial. To avoid very long prompt windows that may exceed the maximum limit, we truncate the agent’s memory to the last 3 self-reflections (experiences).

To avoid syntactic errors, we provide two domain-specific few-shot trajectories to the agent. We use the same few-shot trajectory examples as Yao et al. [30] with GPT-3 for the LLM. AlfWorld tasks, ReAct few-shot prompts, and Reflexion examples are included in the appendix.

Results ReAct + Reflexion significantly outperforms ReAct by completing 130 out of 134 tasks using the simple heuristic to detect hallucinations and inefficient planning. Further, ReAct + Reflexion learns to solve additional tasks by learning in 12 consecutive trials. In the ReAct-only approach, we see that performance increase halts between trials 6 and 7.

Analysis A common error in baseline failed AlfWorld trajectories is when an agent thinks that it has possession of an item but does not actually have the item. The agent proceeds to execute several actions in a long trajectory and is not able to backtrack its actions to find the mistake. Reflexion

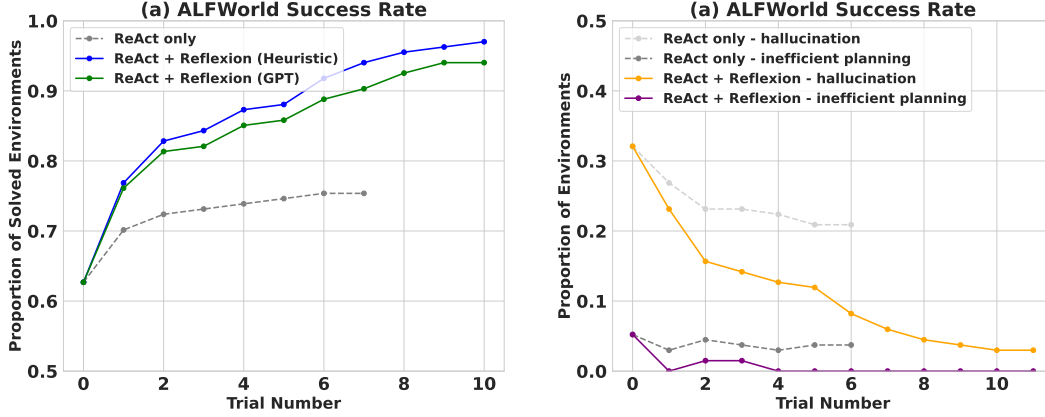


Figure 3: (a) AlfWorld performance across 134 tasks showing cumulative proportions of solved tasks using self-evaluation techniques of (Heuristic) and (GPT) for binary classification. (b) Classification of AlfWorld trajectories by reason of failure.

eliminates almost all of these cases by using self-reflection to distill long, failed trajectories into relevant experiences that can be used as "self-hints" in the future. There are two main cases in which long-term memory helps an agent in AlfWorld: 1) An early mistake in a long trajectory can be easily identified. The agent can suggest a new action choice or even a new long-term plan. 2) There are too many surfaces/containers to check for an item. The agent can exploit its experience memory over several trials to thoroughly search a room. In 3, the learning curve suggests that the learning process occurs over several experiences, meaning that the agent is successfully balancing cases 1 and 2 shown in the immediate spike in the improvement between the first two trials, then a steady increase over the next 11 trials to a near-perfect performance. On the other hand, 3 shows a ReAct-only agent converging at a hallucination rate of 22% with no signs of long-term recovery.

4.2 Reasoning: HotpotQA

HotPotQA [28] is a Wikipedia-based dataset with 113k question-and-answer pairs that challenge agents to parse content and reason over several supporting documents. To test improvement in reasoning *only* ability, we implement Reflexion + Chain-of-Thought (CoT) [26] for step-by-step $Q \rightarrow A$ and $Q, C_{gt} \rightarrow A$ implementations, where Q is the question, C_{gt} is the ground truth context from the dataset, and A is the final answer. Since CoT is not a multi-step decision-making technique, we give C_{gt} to the agent so that we can isolate the reasoning behavior over large sections of the provided text. To test holistic question and answering ability, which requires reasoning and action choice, we implement a Reflexion + ReAct [30] agent that can retrieve relevant context using a Wikipedia API and infer answers using step-by-step explicit thinking. For CoT implementations, we use 6-shot prompting; for ReAct, we use 2-shot prompting, and for self-reflection, we use 2-shot prompting. All examples can be found in the appendix.

Robustly evaluating natural language answers is a long-standing problem in NLP. Therefore, between trials, we use exact match answer grading using the environment to give a binary success signal to the agent. After each trial, the self-reflection loop is employed to amplify the binary signal, similar to the decision-making setup 4.1 in AlfWorld with a memory size of 3 experiences.

Results Reflexion outperforms all baseline approaches by significant margins over several learning steps. Furthermore, ReAct-only, CoT-only, and CoT (GT)-only implementations fail to probabilistically improve on any tasks, meaning that no failed tasks from the first trial from any of the baseline approaches were able to be solved in subsequent trials using a temperature of 0.7 In the Reflexion runs, we allowed the agent to gather experience and retry on failed tasks until it produced 3 consecutive failed attempts on the particular task. Naturally, the CoT (GT) achieved higher accuracy scores as it was given access to the ground truth context of the question. Still, the CoT (GT) agent is unable to correctly infer the correct answer for 39% of the questions, but Reflexion helps the agent to correct its mistakes without access to the ground truth answer to improve its accuracy by 14%.